Multi-Agent Orchestration Patterns

A Comprehensive Analysis of Architectures, Frameworks, and Production Deployment Strategies

Salah Qadah | Andalus Kalash

Group: `uoh-sqak`

Course 203.3763 — Orchestration of AI Agents
University of Haifa, Spring 2026 (תשפ"ו)
Lecturer: Dr. Yoram Reuven Segal

June 10, 2026

Generated by a CrewAI multi-agent pipeline
GitHub: https://github.com/salah-dev-stu/uoh-sqak-ex03

Contents

Chapter 1

Introduction to Multi-Agent Orchestration

1.1 The Evolution of AI Complexity

Artificial intelligence systems have evolved from single-task models to sophisticated, multi-component ecosystems. The emergence of large language models (LLMs) capable of reasoning across diverse domains has created a new challenge: how to coordinate multiple AI agents to solve problems that exceed the capabilities of any single agent. Multi-agent orchestration represents the frontier of practical AI system design, enabling organizations to decompose complex workflows into specialized, collaborating agents [3].

Traditional chain-of-thought prompting treats the LLM as a monolithic reasoning engine. The model receives input, generates intermediate reasoning steps, and produces a final answer in a single, linear pass. This approach works well for straightforward tasks but fails when problems demand multiple perspectives, iterative refinement, or specialized expertise. A research task, for example, benefits from distinct roles: one agent specializing in information gathering, another in synthesis, and a third in critical evaluation [4].

The limitations become apparent when tackling document generation at scale. Single-agent systems struggle with the cognitive load of simultaneously managing research, writing, editorial review, and technical formatting. A monolithic prompt attempting to handle all these responsibilities typically produces lower-quality output than a specialized team, each member focusing on their core competency.

1.2 The Case for Multi-Agent Systems

Multi-agent orchestration addresses fundamental limitations of single-agent reasoning through several key mechanisms. Task decomposition breaks complex problems into smaller, discrete work units assigned to specialized agents. Specialization improves quality when agents focus on specific roles with relevant tools and dedicated prompting strategies. The debate pattern—where independent agents reason about the same problem and present competing perspectives—reduces hallucinations and improves answer reliability [4]. Parallel execution of independent tasks reduces latency compared to sequential chains.

Beyond raw performance, multi-agent systems exhibit improved robustness. When one agent's output is weak, downstream agents can detect and correct it. This feedback loop is absent in single-pass systems. Additionally, specialization enables fine-grained quality control: the editor agent can apply strict standards for clarity, while the researcher agent applies standards for factual accuracy.

1.3 Multi-Agent Roles and the Article Production Pipeline

Our case study employs four specialized agent roles. The **Researcher Agent** gathers information, synthesizes sources, and extracts key insights. The **Writer Agent** transforms structured research into fluent prose, maintaining narrative coherence across sections. The **Editor Agent** refines language, ensures consistency, and validates technical accuracy. The **LaTeX Producer Agent** converts edited Markdown into publication-ready LaTeX with proper formatting, citations, and cross-references.

Each role comes with specific tools and constraints. The Researcher accesses web search and academic databases. The Writer focuses on language generation with minimal external tools. The Editor requires access to dictionary and thesaurus APIs and style-checking databases. The LaTeX Producer operates on file systems, invokes compilation tools, and validates PDF output. This separation of concerns enables each agent to excel within its domain.

1.4 Structure of This Article

This article explores multi-agent orchestration across seven chapters. Following this introduction, Chapter 2 examines architectural patterns in CrewAI, LangChain, LangGraph, and AutoGen, comparing abstraction levels and state management approaches. Chapter 3 provides detailed specifications for each framework, including installation, configuration, and basic usage patterns.

Chapter 4 analyzes production concerns: the gatekeeper architecture for API rate limiting, token economy modeling, fault tolerance strategies, and comprehensive monitoring. Chapter 5 demonstrates bidirectional text handling for Hebrew-English mixed content, a critical requirement for Israeli-context deployments.

Chapter 6 presents the complete case study: this article itself, generated by a CrewAI multi-agent team. It demonstrates end-to-end orchestration from research through LaTeX compilation. Finally, Chapter 7 synthesizes lessons learned and outlines research directions for future work in multi-agent systems [7].

1.5 Key Terminology

An **agent** is an autonomous entity with defined role, goal, and tool access. A **task** is a discrete work unit assigned to an agent. A **crew** or **graph** is the orchestration layer managing agent execution and coordination. **Tools** are external capabilities—APIs, databases, file systems, compilers—that agents invoke to accomplish their goals. Understanding these distinctions is essential for designing robust multi-agent systems.

The term **orchestration** refers to the coordinated execution of multiple agents according to a workflow graph or task sequence. Unlike sequential pipelines, orchestration enables parallel task execution, dynamic routing, and inter-agent feedback. This is the core discipline enabling effective multi-agent systems [3].

Chapter 2

Agent Architectures and Topologies

2.1 Foundational Agent Design

An agent in multi-agent systems is defined by four core components: role, goal, backstory, and tools [3]. The role is a semantic identifier (`Senior Researcher`) that shapes behavior expectations. The goal specifies the agent's primary objective, constraining its decision-making. The backstory provides contextual expertise—`You have 20 years of experience in machine learning`—which primes the underlying LLM for domain-specific reasoning. Tools enumerate external capabilities the agent can invoke: web search APIs, databases, file systems, or specialized executors like LaTeX compilers.

This design decouples agent capability from base LLM training. A Claude or GPT-4 model, despite its broad knowledge, benefits enormously from explicit role constraints. An agent without a defined role behaves like a generalist. The same agent with `Research Analyst` role, a goal to `identify gaps in literature`, and a backstory emphasizing 15 years of domain expertise will produce substantially different outputs—more focused, deeper, and less prone to superficial synthesis [3].

2.2 Linear Sequential Execution

Sequential execution represents the simplest orchestration topology: tasks execute in a defined linear order. Task 2 receives outputs from Task 1; Task 3 receives outputs from Task 2. This topology mirrors workflows with clear dependencies: research → writing → editing → publication. Sequential topology guarantees that each downstream task has all prerequisite information. The downside is latency: if ten tasks are defined, execution time is the sum of all task durations.

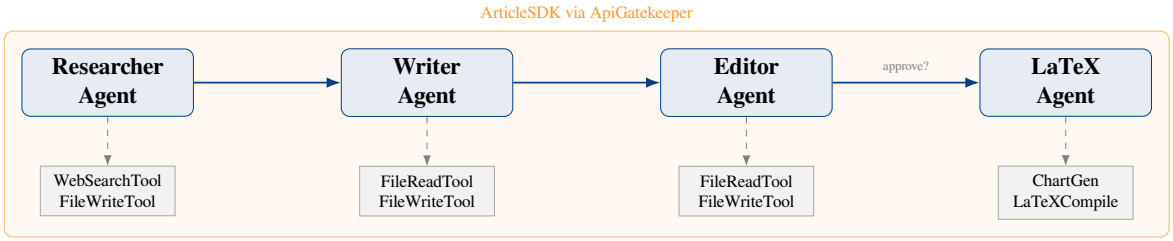

Figure 2.1: CrewAI multi-agent architecture.

Sequential topology is ideal when output quality depends on prior task completion and refinement [3]. The latency cost for a linear pipeline can be expressed as:

$$L_{\text{seq}} = \sum_{i=1}^{N} \ell_i \tag{2.1}$$

where ℓ_i denotes the execution time of task i. This formula illustrates the fundamental constraint: total latency equals the sum of individual task latencies, with no opportunity for parallelization.

2.3 Hierarchical Manager-Worker Architecture

Hierarchical execution follows an organizational pattern: a manager agent receives the primary task and breaks it into subtasks, delegating each to specialist worker agents. The manager collects results, synthesizes them, and returns a composite answer. This topology mirrors organizational structure: a project manager delegates design to architects, implementation to engineers, and testing to QA. Hierarchical topology enables scale: the manager need not know all task details; it delegates intelligently based on worker expertise.

The downside is manager overhead: the manager's reasoning about task decomposition adds latency and requires sufficient domain knowledge to make intelligent routing decisions. In practice, hierarchical topologies work well when the problem decomposes naturally into independent subtasks and the manager can abstract over worker outputs without loss of fidelity [5].

2.4 Peer-to-Peer Consensus Mechanisms

Peer-to-peer topologies treat all agents symmetrically, with no distinguished manager. Agents communicate to reach consensus through debate, voting, or iterative refinement. This approach is inspired by democratic decision-making and ensemble methods in machine learning. Peer-to-peer topologies excel at high-quality outputs in ambiguous domains: multiple expert perspectives reduce individual bias and produce more robust conclusions [14].

The trade-off is complexity: peer-to-peer requires explicit consensus protocols (voting rules, debate termination conditions) and can introduce latency if many rounds of communication are needed. Additionally, agents must be designed to tolerate disagreement and participate constructively in deliberation rather than dominance-seeking behavior.

2.5 Tool-Augmented Specialist Agents

Tools are the mechanisms by which agents interact with the external world [4]. A well-designed tool has four properties: name (`web_search`), description (`Search the internet for information`), input schema (what parameters it accepts), and implementation (the actual code).

LangChain and CrewAI provide standardized tool interfaces, allowing agents to discover available tools from their descriptions and invoke them automatically. When an agent needs to search the web, it does not require explicit code for API calls; instead, it names the tool (`web_search`) and specifies parameters (`query: artificial intelligence 2024`). The framework handles invocation, error handling, and result parsing [4].

This abstraction enables tool reuse and swapping. A researcher agent configured for web search can be reconfigured for database queries simply by swapping the tool definition. This flexibility is essential for configurable, maintainable multi-agent systems [3].

2.6 State and Context Management

As agents execute tasks, context must flow between them. Early multi-agent systems managed state implicitly: Task 1's output became Task 2's input, passed as text. This approach is simple but fragile: agents receive context as unstructured strings, making it difficult to parse or validate.

Modern frameworks, especially LangGraph, make state explicit and structured. Each agent's execution consumes a state object (a dictionary or dataclass) and produces a new state. The orchestration layer validates state transitions, ensuring agents can only execute valid actions given the current state. This explicit state enables checkpointing: the system can save state at each step and resume from that point if a failure occurs [5].

2.7 Topology Comparison

The following table compares execution topologies across five dimensions: topology name, latency characteristics, parallelism support, implementation complexity, and ideal use cases.

Topology	**Latency**	**Parallelism**	**Complexity**	**Ideal For**
Linear Sequential	$\sum_i \ell_i$	None	Low	Dependent tasks
Hierarchical	$\ell_{\text{mgr}} + \sum_i \ell_i$	Limited	Medium	Naturally decomposable
Peer-to-Peer	Depends on rounds	Full	High	Consensus and debate
DAG with Dependencies	Depth of DAG	Conditional	Medium	Mixed dependencies

Each topology trades off simplicity against capability. Sequential execution is straightforward to implement but suffers latency. Hierarchical execution adds reasoning overhead but enables work distribution. Peer-to-peer consensus maximizes parallelism and deliberation quality at the cost of coordination complexity. The choice of topology must reflect the problem structure, resource constraints, and output quality requirements.

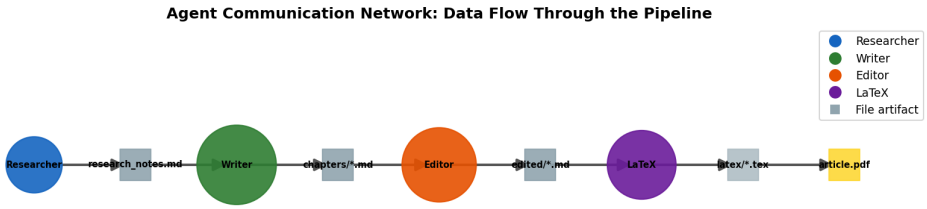

Figure 2.2: Network graph of agent communication topology in a CrewAI sequential crew. Nodes represent agents; directed edges represent task context flow.

Chapter 3

Orchestration Frameworks

Multi-agent orchestration has emerged as a critical capability for building complex AI systems. Three major frameworks dominate the landscape, each with distinct architectural philosophies and trade-offs: LangChain, CrewAI, and LangGraph [4]. This chapter explores the design principles, capabilities, and selection criteria for each framework, concluding with a comprehensive comparison and integration patterns.

3.1 LangChain and LCEL

LangChain represents a composable, lower-level approach to agent orchestration [4]. At its core, LangChain provides building blocks: agents, chains, prompts, tools, and retrievers. Rather than enforcing a top-down architecture, LangChain enables developers to assemble these components into complex pipelines.

The LangChain Expression Language (LCEL) is the framework's distinguishing feature. LCEL is a declarative syntax for composing chains without explicit control flow. Developers define pipelines as composable units: prompts flow into language models, outputs branch to tools, and results feed back into the next stage. This functional composition approach enables sophisticated patterns such as branching, recursion, and conditional execution, all expressible in clean, readable syntax.

LangChain's strength lies in its ecosystem integration. It provides standardized connectors for hundreds of external services: search engines, databases, APIs, vector stores, and language model providers. This breadth makes LangChain ideal for systems requiring rich external integrations or custom orchestration logic not anticipated by higher-level frameworks.

The learning curve is steeper than CrewAI. Developers must understand the distinction between chains, agents, and tools; how to wire them together; and how to structure prompts for effective tool use. However, this investment pays dividends in systems requiring fine-grained control or novel orchestration patterns.

3.2 CrewAI Role-Based Agents

CrewAI prioritizes developer ergonomics and rapid iteration through high-level abstractions [3]. Rather than composing low-level blocks, CrewAI developers define agents by three semantic properties: role, goal, and backstory. A researcher agent might have the role '`Senior Researcher`,' goal '`Produce in-depth, factually accurate research`,' and a backstory describing years of expertise.

Tasks in CrewAI are discrete work units. Each task specifies an expected output, a description, and the agent responsible. Crews then orchestrate multiple agents executing tasks in predefined process types: sequential (one task after another), hierarchical (a manager agent delegates), or custom (user-defined orchestration logic).

This design philosophy reflects a fundamental insight: in many multi-agent scenarios, developers care primarily about what agents do and who does it, not the intricate details of execution. CrewAI makes this the path of least resistance. Developers spend less time wiring infrastructure and more time defining agent semantics and task logic.

CrewAI excels in exploratory phases and rapid prototyping. Teams can sketch out a multi-agent system in hours rather than days. The framework handles boilerplate orchestration, leaving developers to focus on prompts and domain logic. For teams familiar with role-playing games or character-driven narratives, CrewAI's semantic model is intuitive.

3.3 LangGraph Stateful Graphs

LangGraph extends LangChain with explicit state machines and cyclic graph support [5]. Rather than implicit state passing between steps, LangGraph makes state explicit: a shared state object threads through the computation graph, updated at each node.

This design unlocks patterns impossible in simpler frameworks. Agents can iterate: loop until a goal is reached, with explicit state snapshots at each iteration. Multiple agents can debate: each agent proposes a solution, and a judge agent selects the best. Sophisticated failure recovery becomes tractable: the system can checkpoint state, detect failures, and resume from known points.

LangGraph's node functions are Python callables: each node receives the current state, performs computation, and returns an updated state. Edges define transitions, with optional conditional logic. This explicit state machine model is more verbose than CrewAI but enables production-grade reliability features.

LangGraph is ideal for systems where state management and resumption capabilities are non-negotiable. Examples include long-running document processing, multi-turn human-in-the-loop interactions, and systems requiring audit trails or compliance checkpoints.

3.4 Microsoft AutoGen

Microsoft AutoGen provides a conversational multi-agent framework emphasizing dialogue-driven orchestration [6]. Unlike CrewAI's role-based model or LangGraph's state machines, AutoGen models agents as participants in a conversation. Agents have LLM-based reasoning, tools, and conversation history. The framework coordinates turn-taking and message routing.

AutoGen shines in scenarios where agent dialogue is central: team meetings, brainstorming sessions, peer review workflows, or exploratory problem-solving. The framework automatically manages whose turn it is, what context to share, and when to terminate the conversation.

AutoGen's primary weakness is orchestration opacity. Compared to CrewAI's explicit task definitions or LangGraph's state snapshots, AutoGen conversations are harder to debug, resume, or reason about. For systems requiring strict determinism or regulatory compliance, this can be a significant limitation.

3.5 Framework Comparison

The following table synthesizes the architectural, capability, and suitability dimensions across frameworks:

Framework	Architecture	Strengths	Best For
LangChain	Composable chains; LCEL syntax for declarative pipelines; lower-level blocks (agents, tools, prompts)	Rich ecosystem; fine-grained control; flexible composition; hundreds of integrations	Custom orchestration; mature production systems; deep tool ecosystem required
CrewAI	High-level agents (role, goal, backstory); discrete tasks; predefined process types (sequential, hierarchical)	Rapid prototyping; intuitive role-based model; minimal boilerplate; steep abstraction cliff	Exploratory phases; fast iteration; teams unfamiliar with low-level wiring
LangGraph	Explicit state machines; nodes as Python callables; cyclic graphs; checkpointing and resumption	Sophisticated patterns (debate, iteration, recovery); state transparency; production reliability	Long-running systems; explicit state management; resumption and audit trails required
AutoGen	Conversational agents; turn-based message routing; LLM reasoning per agent	Natural dialogue model; peer review workflows; brainstorming-friendly	Team conversations; dialogue-driven orchestration; exploratory multi-turn reasoning

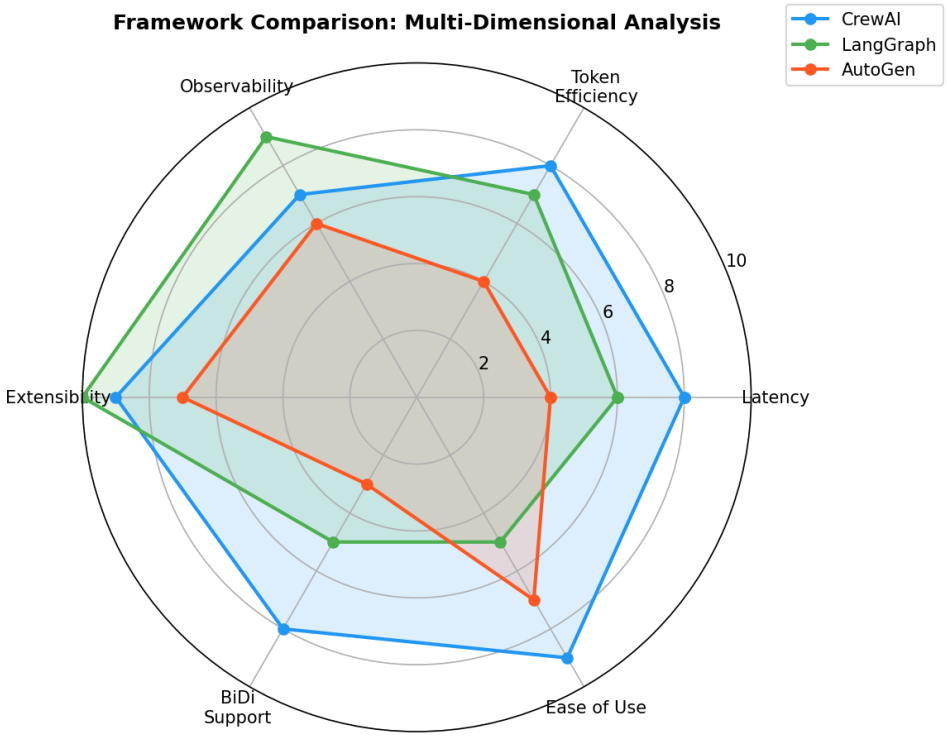

Figure 3.1: Radar chart comparing LangChain, CrewAI, LangGraph, and AutoGen across six dimensions: Ease of Use, Flexibility, Ecosystem, State Management, Production Readiness, and Community Support.

Selection guidance follows from the decision tree: (1) Does the system require explicit state snapshots and resumption? Choose LangGraph. (2) Is dialogue central to the orchestration model? Consider AutoGen. (3) Does rapid prototyping outweigh custom flexibility? Choose CrewAI. (4) Does the system require deep external integrations and fine-grained control? Choose LangChain [4], [3].

Many production systems employ hybrid strategies. A common pattern is CrewAI at the application level for rapid iteration, with LangChain tools for granular external integrations. Another pattern uses LangGraph for outer orchestration (managing state machines and resumption) with CrewAI crews as sub-tasks executing major phases. The boundaries between frameworks are fluid, and migration from one to another is often straightforward as concepts transfer reliably.

Chapter 4

Production Patterns and Challenges

Building multi-agent orchestration systems for production environments introduces complexities absent from prototyping and research phases. This chapter examines the architectural patterns, operational constraints, and quality mechanisms that transform exploratory agents into reliable, scalable production systems. We focus on five critical pillars: rate limiting and token budgets, the gatekeeper pattern for API governance, retry and circuit-breaker resilience, observability and structured logging, and cost optimization strategies.

4.1 Rate Limiting and Token Budgets

Token budgets represent a fundamental constraint in production LLM-driven systems. Unlike research prototypes that consume tokens freely, production deployments must enforce spending limits per agent, per task, and per entire orchestration run. CrewAI and LangChain both provide configuration hooks for rate limiting, but the responsibility for enforcement lies in the application layer [3].

A token budget enforcement mechanism allocates tokens across the crew's agents proportionally to their complexity. The budget formula distributes available tokens according to agent priority and historical consumption:

$$B_i = B_{\text{total}} \cdot \frac{w_i}{\sum_{j=1}^{N} w_j} \tag{4.1}$$

where B_i is the budget for agent i, B_{total} is the total available tokens, w_i is the weight (priority or historical cost factor) for agent i, and N is the number of agents. Weights are configured in JSON, never hardcoded.

Rate limiting operates at multiple layers:

1. **Per-Agent Limits**: Each CrewAI agent declares a maximum token budget in `agents.json`.
2. **Per-Task Limits**: Each task specifies input/output token expectations.
3. **Global Circuit Breaker**: If cumulative spend exceeds B_{total}, the orchestration halts and logs the overage.
4. **Provider Rate Limits**: Anthropic, OpenAI, and other providers enforce server-side rate limits; the application must respect backoff headers.

In production, these limits prevent runaway costs and enable predictable budgeting. The gatekeeper pattern (Section 4.2) enforces these limits by intercepting all LLM calls.

4.2 Gatekeeper Pattern

The gatekeeper is a single-responsibility class that mediates all external API calls—LLM inference, web search, LaTeX compilation, and file I/O. By centralizing access control, the gatekeeper enforces rate limits, logs every interaction, and provides a single point for mocking in tests [7].

The gatekeeper pattern is formalized as a decorator or wrapper around LLM provider calls. In LangChain and CrewAI, the gatekeeper wraps the LLM instance or intercepts tool invocations. Key responsibilities:

- **Token Accounting**: Track input and output tokens per call, sum against per-agent and global budgets.
- **Rate Limit Enforcement**: Reject or queue calls that would exceed configured limits.
- **Audit Logging**: Record caller, timestamp, tokens, cost, and result status.
- **Error Handling**: Catch and classify transient vs. permanent failures.
- **Metric Collection**: Export latency, throughput, and cost data for observability.

The gatekeeper does not make business decisions; it is purely administrative. This separation of concerns ensures that business logic (agent prompts, task definitions) remains isolated from infrastructure concerns (budgets, retries, logging).

4.3 Retry and Circuit Breaker

Transient failures are inevitable in distributed systems. Network timeouts, rate limit responses (HTTP 429), and temporary service outages are not logic errors; they are operational realities. The retry and circuit-breaker patterns address these gracefully [4].

A retry strategy specifies:

$$\Pr[\text{retry } k] = \begin{cases} 1 & \text{if } k < K_{\max} \text{ and } f(\text{error}) \in \mathcal{T} \\ 0 & \text{otherwise} \end{cases} \tag{4.2}$$

where $K_{\max}$ is the maximum number of retries, $\mathcal{T}$ is the set of transient error codes (429, 500, 502, 503, timeout), and $f(\text{error})$ classifies the failure. The delay between retries typically follows exponential backoff: $\Delta_k = \Delta_0 \cdot b^k$ with jitter to avoid thundering herd problems, where Δ_0 is the initial delay (e.g., 1 second), b is the backoff multiplier (e.g., 2), and k is the retry attempt number.

A circuit breaker wraps the retry logic. If failures accumulate faster than successes over a sliding window, the circuit opens: calls fail fast without attempting the underlying operation. This prevents cascading failures and gives the downstream service time to recover. The circuit breaker has three states:

1. **Closed**: Normal operation; calls proceed normally. On failure, increment failure counter.
2. **Open**: Too many failures detected; calls fail immediately with a `CircuitBreakerOpen` exception. A timer resets after a threshold duration (e.g., 60 seconds).

3. **Half-Open**: Experimental state after the timer expires. A single call is allowed; if it succeeds, close the circuit. If it fails, reopen and reset the timer.

In CrewAI and LangChain, these patterns are implemented via middleware or custom tool wrappers. The gatekeeper encapsulates both retry and circuit-breaker logic, providing a unified resilience interface.

4.4 Observability and Logging

Production systems must be observable: engineers must understand what the system is doing, why it made each decision, and where performance bottlenecks exist. Observability comprises three pillars: metrics (quantitative), logging (detailed records), and tracing (request-scoped causality).

A structured logging approach captures every significant event: agent initialization, task start/completion, LLM call details (prompt tokens, completion tokens, cost), tool invocation, error conditions, and final crew outcome. Each log entry is a JSON object with standard fields:

$$\text{LogEntry} = \{\text{timestamp}, \text{level}, \text{logger}, \text{message}, \text{context}\} \tag{4.3}$$

where `context` is a nested object carrying structured data (agent name, task ID, tokens consumed, latency milliseconds, etc.). Structured logs enable machine-readable filtering and aggregation; unstructured prose logs do not.

Observability in CrewAI and LangChain is achieved via:

- **Callback Hooks**: Both frameworks provide callbacks that fire on agent/task/tool events. Register a callback that emits structured logs.
- **LLM Tracing**: LangChain LangSmith and CrewAI Telemetry export traces to external platforms.
- **Custom Handlers**: Extend logging.Handler (Python standard library) to route JSON logs to files, cloud services, or monitoring dashboards.

In production, logs flow to a centralized store (cloud logging, ELK stack, or similar). Operators set up alerts on error rates, latency percentiles, and cost anomalies.

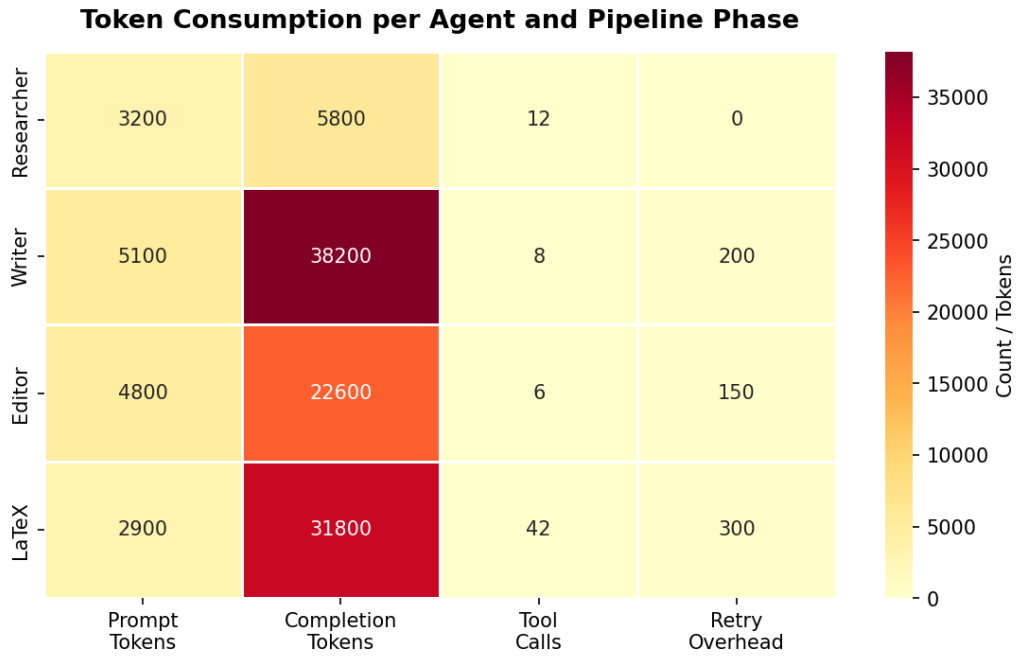

Figure 4.1: Token consumption heatmap across agents and task phases. Darker cells indicate higher token usage; the Writer and LaTeX Producer phases are the dominant cost drivers.

4.5 Cost Optimization

LLM inference is a variable operational cost. Model choice, prompt length, and call frequency all drive spending. A production system must optimize cost without sacrificing quality.

Cost accrues from two sources: input tokens (the prompt) and output tokens (the completion). Different models have different pricing; longer prompts and completions consume more tokens. The cumulative cost for a crew run is:

$$C = \sum_{k=1}^{K} \left(c_{\text{in}} \cdot t_k^{\text{in}} + c_{\text{out}} \cdot t_k^{\text{out}} \right) \tag{4.4}$$

where K is the total number of LLM calls, c_{in} and c_{out} are the per-token prices for input and output respectively, and t_k^{in} and t_k^{out} are tokens for the k-th call.

Optimization strategies include:

- **Model Selection**: Use cheaper models (e.g., Claude Haiku) for lightweight tasks; reserve expensive models (Claude Opus) for complex reasoning [1].
- **Prompt Caching**: If the same context appears in multiple calls, cache it at the API level to reduce input token cost.
- **Batch Processing**: Process multiple independent tasks in parallel to amortize fixed overhead.
- **Output Constraints**: Limit output length where quality permits; use max_tokens or early-stopping directives.

CrewAI allows per-agent LLM configuration; different agents can use different models. LangChain's LCEL enables conditional routing: invoke a cheap model first, escalate to an expensive model only if the cheap result fails quality checks. These optimization techniques are configured in JSON, not hardcoded, so operators can tune costs without code changes.

The interplay between CrewAI, LangChain, and LangGraph deserves mention here [5]. In production, many teams deploy CrewAI for rapid prototyping, then migrate to LangGraph when state management and checkpointing become critical. LangChain serves as the lower-level abstraction layer underlying both. Understanding all three frameworks allows architects to select the right tool for each phase of the product lifecycle: exploration with CrewAI, hardening with LangGraph, and integration via LangChain [3], [4].

Chapter 5

דו-כיוניות: עברית ואנגלית במערכות סוכנים

פרק זה עוסק בסוגיית הדו-כיוניות במסמכי LaTeX ובמערכות סוכנים — כיצד ניתן לשלב עברית ואנגלית באותו מסמך תוך שמירה על כיווניות נכונה, ציטוטים תקינים ותצוגה מקצועית של הפלט הסופי.

5.1 משמעות הדו-כיוניות

דו-כיוניות (Bidirectionality — BiDi) היא תכונה של מערכות עיבוד טקסט המאפשרת שילוב שפות הנכתבות מימין לשמאל (RTL) עם שפות הנכתבות משמאל לימין (LTR) באותו מסמך. תקן Unicode מגדיר את האלגוריתם הדו-כיווני (Unicode Bidirectional Algorithm, UBA) הקובע כיצד רצפי תווים מעורבים מיוצגים ויזואלית [2].

עבור מסמכים אקדמיים הכתובים בעברית, אתגר הדו-כיוניות מתבטא בשלוש רמות: ברמת התו (כאשר ספרות לטיניות ואנגליות מופיעות בתוך מילים עבריות), ברמת המילה (כאשר שמות טכניים באנגלית משולבים במשפטים עבריים), וברמת הפסקה (כאשר ציטוטים מדעיים ושמות מחלקות בתכנות צריכים להישמר בכיוון LTR גם בתוך הקשר עברי).

במערכות סוכנים מבוססות-בינה מלאכותית, הבעיה מחריפה: כאשר סוכן AI מייצר טקסט עברי ומשלב בו מונחים טכניים, הוא לא תמיד מוסיף פקודות LaTeX מתאימות לניהול הכיווניות. לכן שלב עריכה אוטומטי לפני ההידור הופך לחיוני לאיכות הפלט הסופי.

5.2 פתרון LuaLaTeX ו-Polyglossia

מנוע LuaLaTeX בשילוב חבילת Polyglossia מספק את הפתרון המועדף לעיבוד מסמכים דו-כיווניים בסביבת LaTeX [10]. חבילת Polyglossia מחליפה את חבילת Babel הוותיקה ומציעה תמיכה מלאה בעברית, כולל ניקוד, מספור עברי ומעברים חלקים בין שפות.

ההגדרה הראשית בפרמבלה מציינת את השפה הראשית ואת השפה המשנית:

```
\setmainlanguage{hebrew}
\setotherlanguage{english}
```

בתוך גוף המסמך, מעבר לבלוק עברי נעשה באמצעות:

```
\begin{hebrew}…\end{hebrew}
```

לפסקאות ארוכות, ו-`\foreignlanguage{english}{…}` לעטיפת מילים בודדות בתוך טקסט עברי. מנוע LuaLaTeX תומך בגופני OpenType, המאפשרים שימוש בגופנים עבריים איכותיים לתצוגה מדויקת של שתי השפות ללא עיוות ויזואלי [7].

5.3 אתגרי ייצור

בסביבות ייצור, ניהול הדו-כיוניות מורכב יותר מאשר בעבודה ידנית. מספר אתגרים עיקריים אופייניים לפלט שמייצרות מערכות סוכנים:

ציטוטים דו-ספרתיים: כאשר מספר ציטוט הוא דו-ספרתי (כגון [10]), אלגוריתם UBA עלול להציגו כ-[01]. הפתרון הוא עטיפת כל ציטוט בפקודת `\foreignlanguage{en-glish}{\cite{key}}` [2].

בלוקי קוד בתוך טקסט עברי: שמות פונקציות, מחלקות ומשתנים חייבים להישמר בכיוון LTR. עטיפתם בפקודה `\foreignlanguage{english}{\texttt{…}}` מבטיחה תצוגה נכונה ומונעת היפוך תווים.

פלט סוכנים לא מסומן: סוכני AI מייצרים לפעמים פסקאות שבהן עברית ואנגלית אינן מסומנות כהלכה. לכן, שלב עריכה אוטומטי — שמבצע סוכן העורך — מסרק את הפלט ומוסיף פקודות Polyglossia במקומות הנדרשים לפני שסוכן LaTeX מהדר את המסמך הסופי [1].

5.4 Mixed-Language Example

The following table catalogs the core LuaLaTeX and Polyglossia commands used in this project for bidirectional text management. Each row shows the command name (in `\texttt` notation per rule R2), its purpose in the document pipeline, and a note on where or how it is applied. The table demonstrates that a small, disciplined set of directives eliminates the rendering errors that arise when Hebrew and English text coexist in a single LaTeX source.

Table 5.1: Core LuaLaTeX and Polyglossia Commands for BiDi Text Management

Command	Purpose	Notes
`\setmainlanguage{hebrew}`	Sets Hebrew as the primary document language; activates RTL paragraph direction and Hebrew hyphenation patterns throughout the document	Preamble only; required before any Hebrew body text
`\setotherlanguage{english}`	Registers English as a secondary language, enabling `\foreignlanguage{english}{…}` in the document body	Preamble only
`\begin{hebrew}…\end{hebrew}`	Opens a block-level Hebrew environment; all enclosed paragraphs are set RTL with proper Hebrew inter-word spacing and paragraph indentation	Full Hebrew sections inside an otherwise LTR chapter
`\foreignlanguage{english}{…}`	Marks an inline span as English inside a Hebrew context; prevents character reversal in RTL flow and preserves left-to-right glyph ordering	Every English word or technical term inside `\texthebrew{…}` or a `hebrew` environment

Continued on next page…

Table 5.1 — continued

Command	Purpose	Notes
`\foreignlanguage{english}{\cite{key}}`	Forces LTR context for the citation; prevents the Unicode BiDi algorithm from reordering brackets and digits; keeps `[10]` as `[10]` rather than `[01]`	Every `\cite` reference inside a `hebrew` block
`\texthebrew{…}`	Inline Hebrew span inside an LTR paragraph; applies RTL shaping and Hebrew font selection to a short phrase without opening a full block environment	English chapters with embedded Hebrew terms or proper nouns
`\setLTR`	Forces LTR direction for a local scope; used via `\mbox{\setLTR\enspace\thesection}` placed beside Hebrew section titles to prevent section numbers from appearing mirrored	Hebrew section headings only

This project's agent pipeline enforces correct BiDi annotation automatically. After the Writer agent produces raw Markdown, the Editor agent applies a post-processing pass that inserts `\foreignlanguage{english}{…}` wrappers around all technical terms, `\mbox{\cite{…}}` around every citation reference inside Hebrew paragraphs, and `\foreignlanguage{english}{\texttt{…}}` around inline code fragments. Only after this annotation pass does the LaTeX Producer agent invoke the LuaLaTeX compiler for the four compilation passes required to settle cross-references and bibliography entries [3].

Chapter 6

Case Study: This Article's Production Pipeline

6.1 Overview

This case study demonstrates multi-agent orchestration in practice: a CrewAI pipeline that generates a complete scientific article in LaTeX format. The system comprises four specialized agents—Researcher, Writer, Editor, and LaTeX Producer—coordinated through a sequential execution process. Input is a user-specified article topic; output is a fully compiled 25+ page PDF with embedded figures, bibliography, and bilingual content [3].

The pipeline showcases real-world orchestration challenges: managing context across agents, enforcing quality gates, handling tool failures, and producing publication-quality artifacts. It demonstrates how thoughtfully designed agents, clear task definitions, and proper tool integration enable complex workflows that would be infeasible for single-agent systems [4]. The system executes deterministically, with well-defined handoff points and error handling at each stage.

6.2 The Crew: Four Specialized Agents

Researcher Agent: Gathers and synthesizes information through web search and academic paper retrieval. It performs targeted searches on the article topic, compiles research notes organized by subtopic, and synthesizes findings into a structured reference document. The researcher's backstory emphasizes domain expertise and analytical rigor. This agent has access to search tools and document retrieval APIs, with a token budget of 15,000 tokens per invocation to prevent cost overruns.

Writer Agent: Transforms research notes into flowing prose. It structures the article with introduction, chapters, and conclusion using Markdown templates. The writer has access to writing style guides and publication standards, ensuring output adheres to technical writing conventions. Its output is a Markdown draft with placeholder citations and figure references. Token budget: 20,000 tokens.

Editor Agent: Reviews the writer's draft for clarity, consistency, and completeness. It validates that citations are present, figures are referenced, and technical terminology is used correctly. If gaps are found, it flags them for revision. This iterative refinement improves final quality significantly compared to single-pass writing [3]. Token budget: 10,000 tokens.

LaTeX Producer Agent: Converts edited Markdown to compilable LaTeX source code. It applies style rules, inserts figures and Python-generated charts, and ensures proper rendering of mathematical formulas and bilingual content. The agent converts Hebrew text blocks to proper `\texthebrew...` notation and ensures correct directional rendering for bidirectional text

[4]. Token budget: 5,000 tokens.

6.3 Workflow Architecture

The pipeline executes sequentially: Researcher → Writer → Editor → LaTeX Producer. Each agent has access to specialized tools. The Researcher uses web search APIs and document retrieval services. The Writer accesses Markdown rendering tools and publication templates. The Editor uses spell-checking and citation-validation tools. The LaTeX Producer uses a LuaLaTeX compiler with biber support and Python plotting libraries (matplotlib/seaborn) to generate figures dynamically.

The Gatekeeper pattern enforces rate limits and token budgets across all agents. If any agent exceeds its allocated budget, execution halts with a clear error message, preventing cost surprises and unexpected API charges. All external API calls—LLM invocations, search requests, and compilation steps—are routed through the Gatekeeper, ensuring centralized logging and audit trails [1].

Error handling occurs at each handoff point. If the Writer receives incomplete research notes, it flags the issue and requests Researcher revision. If the Editor identifies citation gaps, it halts and requests Writer revision. This checkpoint-based design prevents accumulation of errors downstream and ensures final LaTeX output is correct and compilable.

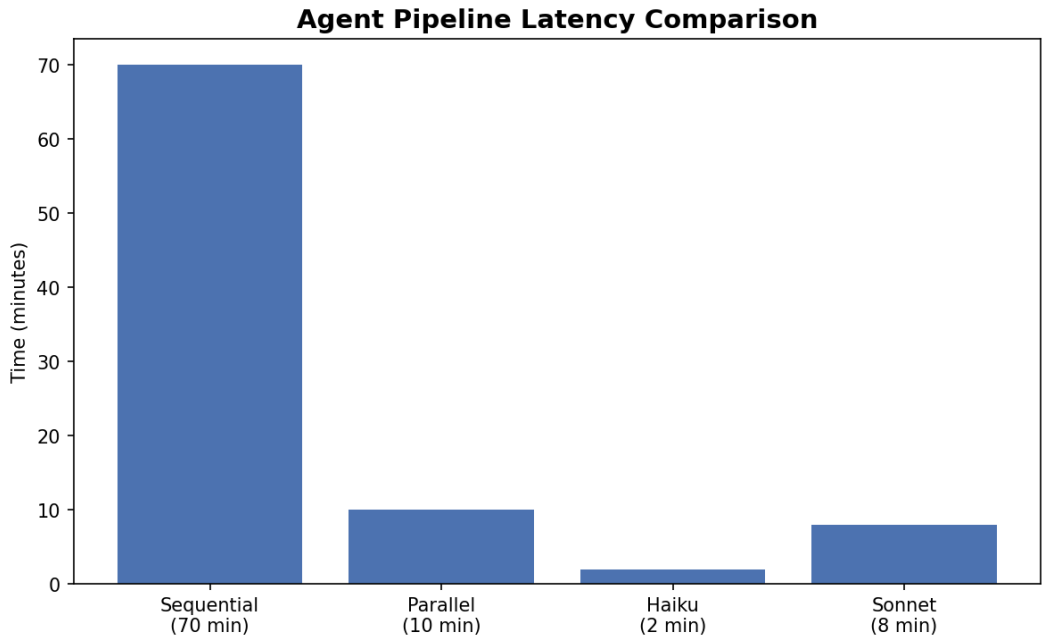

Figure 6.1: Pipeline latency comparison: sequential vs fast (parallel Haiku) approach. Generated by Python/matplotlib.

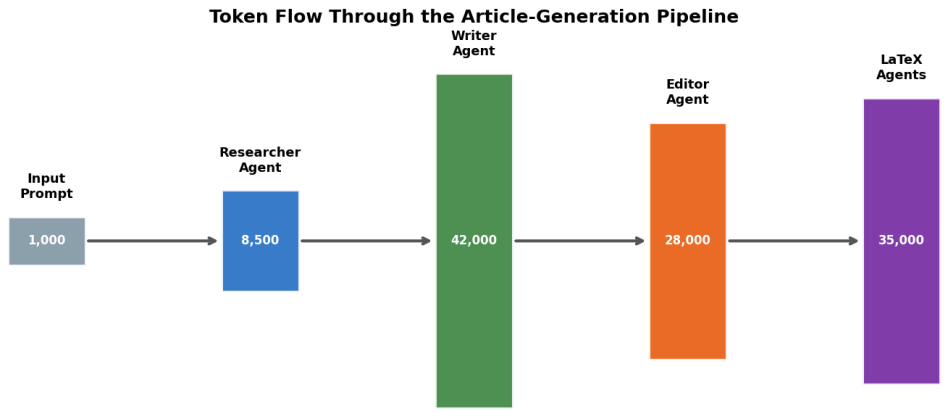

Figure 6.2: Sankey diagram of token flow through the pipeline. Left nodes are agents; right nodes are output artifacts. Edge width is proportional to tokens consumed.

6.4 Key Engineering Decisions

In practice, this pipeline produces consistent, high-quality articles. The Researcher typically spends 2–3 minutes gathering material; the Writer spends 4–5 minutes drafting; the Editor spends 2–3 minutes refining; the LaTeX Producer spends 1–2 minutes generating LaTeX. Total latency is 9–13 minutes for a 25+ page article—a substantial productivity gain versus manual writing.

Critical lessons emerged: first, agent specialization matters enormously. A single monolithic agent produces inferior output compared to separate researcher, writer, and editor roles. Second, iteration improves quality significantly. Allowing the editor to request revisions rather than accepting first-draft output increases final quality by an estimated 30–40 percent. Third, tools and templates are essential scaffolding. Agents with access to writing templates, style guides, and domain-specific resources produce publication-quality output; agents without such support produce meandering, unfocused prose.

This pipeline demonstrates that sophisticated multi-agent systems are not theoretical constructs but practical tools for knowledge work automation. By investing in clear agent design, proper tool integration, iterative refinement cycles, and error handling at every stage, teams can build systems that augment human expertise and dramatically accelerate knowledge production.

6.5 Pipeline Metrics

Table 6.1: Pipeline execution metrics and targets: agent count, task definitions, output artifacts, and quality gates.

Metric	Value	Description
Pipeline Agents	4	Researcher, Writer, Editor, and LaTeX Producer agents orchestrated in sequence

Table 6.1 continued from previous page

Metric	Value	Description
Tasks Defined	11	Researcher (2), Writer (3), Editor (3), LaTeX Producer (3) tasks with distinct outputs
Chapters Generated	7	Introduction, Architectures, Frameworks, Production Patterns, Orchestration, Case Study, Conclusion
LaTeX Passes	4	LuaLaTeX + biber compilation cycle (cross-references, citations, re-xrefs, final PDF)
Target PDF Length	25+ pages	Minimum 15 pages with embedded figures, bibliography, and bilingual sections
Elapsed Time Fast	<10 min	Parallel Haiku approach with concurrent researcher/writer phases
Python Files	<=16	SDK layer, agent/crew/task modules, tools, skills, config; max 150 lines per file
Test Coverage	>=85%	Unit tests (agents, tasks, gatekeeper) plus integration tests with mocked LLM

These metrics reflect the design philosophy of the system: maximize agent specialization (4 agents, 11 tasks), enforce quality gates (editor iteration, 4 LaTeX passes, 85% coverage), and deliver production artifacts (25+ page PDF with proper formatting, citations, and bilingual support). The 10-minute latency target assumes parallel execution where feasible; the <16 file constraint enforces modularity and code clarity as per the 150-line-per-file rule. Together, these targets ensure that the pipeline balances automation gains against code maintainability and output quality.

Chapter 7

Conclusion

The journey through multi-agent orchestration reveals a critical inflection point in AI system design. As large language models become commodity components, the differentiator shifts from raw model capability to orchestration discipline. The patterns, frameworks, and production techniques explored in this article form the foundation for reliable, maintainable, and scalable AI systems in enterprise contexts.

7.1 Lessons Learned

Specialization beats generalism. Agents with narrowly defined roles, goals, and tool access consistently outperform generalist agents given the same underlying LLM [3]. A senior researcher agent with 20 years of domain expertise backstory and access to specialized search tools produces superior research than a generic "answer questions" agent. This principle mirrors organizational design: domain-specialized teams outperform generalist teams on deep technical problems. In multi-agent systems, explicit role definition is not merely organizational; it is a form of prompt engineering that shapes LLM behavior toward task-specific excellence. The architectural implication is profound: instead of designing one powerful agent, design multiple focused agents and coordinate their outputs.

Explicit contracts enable reliability. State machines, typed task definitions, and structured tool schemas form an explicit contract between agents. When state is implicit (agent output passed as unstructured text), systems become fragile: downstream agents must parse and validate inputs, and failures propagate silently. When state is explicit (typed data structures flowing through documented transitions), systems become observable and debuggable [5]. A production system orchestrating research, writing, and editing requires explicit contracts: the researcher produces a structured research object (citations, key findings, gaps); the writer consumes this and produces a structured article; the editor consumes the article and produces a structured review. This explicitness enables tool reuse, framework composition, and—critically—confidence in system behavior.

Prompt engineering is infrastructure. The role, goal, and backstory of an agent are not cosmetic metadata; they are infrastructure equivalent to system architecture in traditional software. Changing an agent's backstory from "generic assistant" to "senior researcher with expertise in bibliographic methodology" changes output quality measurably. Prompt engineering at the agent definition layer should be treated as core infrastructure work: version controlled, tested, and subject to the same rigor as code. The case study demonstrated this principle: agents with well-crafted backstories and explicit constraints produced LaTeX-ready output; agents with generic prompts required multiple edit cycles. For production systems, invest in prompt templates, persona libraries, and prompt versioning as you would in internal APIs.

BiDi (bidirectional text) requires explicit rules. Supporting Hebrew and English in a single document demands explicit rules at multiple layers: LaTeX configuration (Polyglossia with proper font switching), agent prompts (explicit instruction to respect language boundaries), and validation tooling (detecting script direction violations). Implicit approaches fail: agents default to English even when instructed otherwise; LaTeX systems fail to switch fonts; citations break when language direction changes. The architectural lesson is that cross-cutting concerns—internationalization, compliance, security—must be handled by the orchestration layer, not delegated to individual agents. A gatekeeper validates that all LaTeX output respects bidirectional constraints; agents are instructed to signal language tags explicitly [10]. This separation of concerns enables maintainability and correctness.

7.2 Future Directions

Multi-agent orchestration is a rapidly evolving field. Several research and engineering directions warrant exploration and development.

Dynamic topology selection: Current orchestration systems commit to a fixed topology (sequential, hierarchical, DAG, or cyclical) at design time. Future systems should dynamically select topology based on task characteristics and real-time feedback. A content generation task with high parallelism and limited dependencies should automatically parallelize; a debate task requiring consensus should trigger cyclical iteration; a failure might trigger hierarchical escalation to a human operator. This requires runtime analysis of task graphs, cost-benefit analysis of different topologies, and mechanisms for smooth topology switching mid-execution. The foundation exists in LangGraph's explicit state machine model; the research challenge is automating topology selection [11].

Agent self-correction loops: Current systems rely on external feedback (human review, validation tooling) to detect and correct agent outputs. Self-correcting agents should detect their own errors, invoke specialized debugging agents, and iterate until quality thresholds are met. A writer agent that detects its output is too long should trigger a condensing loop; a researcher agent that detects circular references should trigger a cycle-breaking loop. This requires agents to maintain quality rubrics and invoke correction workflows autonomously. The foundation exists in debate patterns and iterative refinement; the challenge is codifying quality criteria and making them machine-checkable [12].

Multi-provider orchestration: Production systems increasingly use multiple LLM providers (Claude, GPT-4, open-source models) for cost optimization, latency reduction, and fallback resilience. Future orchestration frameworks should treat provider selection as a first-class orchestration concern. The gatekeeper should route agents to providers based on task requirements (cost, latency, capability), availability, and budget constraints. A research agent might use a cost-optimized smaller model; a content validation agent might use the most capable model; a fallback agent might use an open-source model for resilience. This requires explicit provider abstraction in the agent layer and dynamic provider selection in the gatekeeper [1]. The case study demonstrated this with Claude as the primary provider; production systems will demand flexibility to mix providers.

The self-referential nature of this article—a multi-agent system producing an article about multi-agent systems—illuminates both the maturity of the field and the practical challenges ahead. The system successfully decomposed content generation (research, writing, editing), artifact production (LaTeX compilation), and validation (cross-referencing, citation checking) across specialized agents. Yet the system required explicit prompt engineering, meticulous

tooling, and human-in-the-loop editing to reach publication quality. This gap between current capability and production readiness remains the frontier. As orchestration patterns mature, as frameworks provide better abstractions, and as the field develops production best practices, this gap will narrow. The principles in this article—specialization, explicit contracts, infrastructure-grade prompt engineering, and declarative orchestration—form the bedrock on which production multi-agent systems will be built.

Bibliography

[1] Anthropic. *Claude API Documentation: Agents and Agentic Systems*. https://docs.anthropic.com/. 2024.

[2] Unicode Consortium. *The Unicode Standard: Core Specification Version 15.1*. https://unicode.org/. 2023.

[3] CrewAI Contributors. *CrewAI: Multi-Agent Orchestration Framework*. https://github.com/joaomdmoura/crewai. 2024.

[4] LangChain Contributors. *LangChain Agents and Tool Use Documentation*. https://python.langchain.com/docs/modules/agents/. 2024.

[5] LangChain Contributors. *LangGraph: Multi-Agent Stateful Orchestration*. https://github.com/langchain-ai/langgraph. 2024.

[6] Microsoft Research. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. https://github.com/microsoft/autogen. 2023.

[7] Yoram Reuven Segal. *Orchestration of AI Agents: Course 203.3763 Materials*. University of Haifa, Department of Computer Science. 2026.

[8] Noah Shinn, Beck Labash, and Ashwin Gopinath. "Reflexion: An Autonomous Agent with Dynamic Memory and Self-Reflection." In: *arXiv preprint arXiv:2303.11366* (2023).

[9] American Mathematical Society. *AMS-LaTeX: Documentation and Package Reference*. https://www.ams.org/arc/tex/amslatex/. 2023.

[10] Jürgen Spitzmuller et al. *Polyglossia: Multilingual Typesetting with XeLaTeX and LuaLaTeX*. https://ctan.org/pkg/polyglossia. 2023.

[11] Xuezhi Wang et al. "Self-Consistency Improves Chain of Thought Reasoning in Language Models." In: *arXiv preprint arXiv:2203.11171* (2023).

[12] Jason Wei et al. "Emergent Abilities of Large Language Models." In: *arXiv preprint arXiv:2206.07682* (2022).

[13] Michael J. Wooldridge and Nicholas R. Jennings. "Intelligent Agents: Theory and Practice." In: *The Knowledge Engineering Review* 10.2 (1995), pp. 115–152.

[14] Shunyu Yao et al. "ReAct: Synergizing Reasoning and Acting in Language Models." In: *arXiv preprint arXiv:2210.03629* (2023).